

Bitwise Operations Complete DSA Notes

1. What are Bitwise Operations?

Operations directly on binary bits.

Used in:

- Competitive programming
- Optimization
- Subsets
- DP
- Low level systems

2. AND (&)

Both bits must be 1.

Example:

101
011

001

Uses:

Checking bits and masks.

3. OR (|)

Any bit 1 gives output 1.

Used for:

Setting bits.

4. XOR (^)

Different bits give 1.

Properties:

$$x \oplus x = 0$$

$$x \oplus 0 = x$$

Used for finding unique elements.

5. NOT (~)

Flips bits:

$$0 \rightarrow 1$$

$$1 \rightarrow 0$$

6. Left Shift

$$x \ll k$$

means:

$$x * 2^k$$

7. Right Shift

$$x \gg k$$

means:

$$x / 2^k$$

8. Important Bit Tricks

Check ith bit:

$$n \& (1 \ll i)$$

Set ith bit:

$$n \mid (1 \ll i)$$

Clear ith bit:

$n \& \sim(1 <$

Toggle:

$n \wedge (1 <$

9. Brian Kernighan Algorithm

Used to count set bits.

Operation:

$n = n \& (n-1)$

Removes last set bit.

Complexity:

$O(\text{number of set bits})$

10. Power of Two

A power of two has exactly one set bit.

Formula:

$n \& (n-1) == 0$

11. Bitmasking

Use binary numbers to represent choices.

For n elements:

total subsets:

2^n

Used in:

- Subsets

- DP
- Graph states

Python Implementation

```
def check(n,i):
    return n&(1<<i)

def setBit(n,i):
    return n|(1<<i)

def clearBit(n,i):
    return n& ~(1<<i))

def toggle(n,i):
    return n^(1<<i)

def countBits(n):
    count=0
    while n:
        n=n&(n-1)
        count+=1

    return count
```

C++ Implementation

```
bool check(int n,int i){
    return n&(1<<i);
}

int setBit(int n,int i){
    return n|(1<<i);
}

int clearBit(int n,int i){
    return n& ~(1<<i));
}

int toggle(int n,int i){
    return n^(1<<i);
}
```

Java Implementation

```

boolean check(int n,int i){
return (n&(1<<i))!=0;
}

int setBit(int n,int i){
return n|(1<<i);
}

int clearBit(int n,int i){
return n&~(1<<i);
}

int toggle(int n,int i){
return n^(1<<i);
}

```

Generate All Subsets Using Bitmask

```

def subsets(nums):
    n=len(nums)
    ans=[]

    for mask in range(1<<n):
        temp=[]

        for i in range(n):
            if mask&(1<<i):
                temp.append(nums[i])

        ans.append(temp)

    return ans

```

Advanced Bit Manipulation Topics

1. Bitmask DP
2. XOR Trie
3. Gray Code
4. Binary Lifting
5. Fenwick Tree

6. Fast exponentiation

Important Leetcode Problems

LC136 Single Number

LC137 Single Number II

LC191 Number of 1 Bits

LC231 Power of Two

LC268 Missing Number

LC78 Subsets

LC338 Counting Bits

LC421 Maximum XOR

Complexities

Basic operations:

$O(1)$

Counting bits:

$O(\text{set bits})$

Subsets:

$O(n \cdot 2^n)$